AI Assisted Coding

Annapu Vasantha Shoba Rani

2303A51395

Lab Assignment 20.3 – Security Testing

Task 1: Password Strength & Validation

Question

Analyze and improve password validation in a user registration system.

Prompt

"Generate a Python user registration script and improve it with strong password validation rules using regex and constraints."

Code

```
import re


def is_strong_password(password):
    if len(password) < 8:
        return False
    if not re.search(r"[A-Z]", password):
        return False
    if not re.search(r"[a-z]", password):
        return False
    if not re.search(r"[0-9]", password):
        return False
    if not re.search(r"[@$!%*?&]", password):
        return False
    return True


password = input("Enter password: ")
```

```
if is_strong_password(password):
    print("Password is strong")
else:
    print("Weak password")
```

Output

Enter password: Test@123

Password is strong

Code Explanation

- **Checks minimum length (8)**
- **Uses regex for uppercase, lowercase, number, special char**
- **Prevents weak passwords**
- **Improves security**

Task 2: Secure File Upload

Question

Secure a file upload system by validating file type and size.

Prompt

"Generate Flask file upload code with validation for file type, size limit, and secure storage."

Code

```
from flask import Flask, request, jsonify
import os

app = Flask(__name__)

UPLOAD_FOLDER = 'uploads'
```

```
ALLOWED_EXTENSIONS = {'txt', 'pdf'}
MAX_SIZE = 2 * 1024 * 1024  # 2MB

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files['file']

    if not allowed_file(file.filename):
        return jsonify({"error": "Invalid file type"}), 400

    if len(file.read()) > MAX_SIZE:
        return jsonify({"error": "File too large"}), 400

    file.seek(0)
    filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
    file.save(filepath)

    return jsonify({"message": "File uploaded successfully"})

app.run(debug=True)
```

Output

POST /upload →

{"message": "File uploaded successfully"}

Code Explanation

- Validates file extension
- Limits file size (2MB)
- Stores file in safe directory
- Prevents malicious uploads

Task 3: API Security Review

Question

Review and secure an API endpoint.

Prompt

"Analyze Flask API and improve it using authentication, validation, and rate limiting."

Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)

API_KEY = "secure123"

@app.route('/data', methods=['GET'])
def get_data():
    key = request.headers.get('API-Key')

    if key != API_KEY:
        return jsonify({"error": "Unauthorized"}), 401

    return jsonify({"data": "Secure Data"})

app.run(debug=True)
```

Output

GET /data →

{"data": "Secure Data"}

Code Explanation

- **Adds API key authentication**
- **Prevents unauthorized access**
- **Uses header validation**
- **Improves endpoint security**

Task 4: XSS Prevention

Question

Prevent XSS attacks in a feedback form.

Prompt

"Generate HTML form and secure it against XSS using escaping techniques."

Code

```
<!DOCTYPE html>
<html>
<body>

<form method="POST">
  <input type="text" name="feedback">
  <button type="submit">Submit</button>
</form>

<script>
function escapeHTML(str) {
   return str.replace(/&/g, "&amp;")
```

```
            .replace(/</g, "&lt;")
            .replace(/>/g, "&gt;");
}

document.querySelector("form").onsubmit = function(e) {
   e.preventDefault();
   let input = document.querySelector("input").value;
   let safe = escapeHTML(input);
   document.body.innerHTML += "<p>" + safe + "</p>";
};
</script>

</body>
</html>
```

Output

Input: <script>alert(1)</script>

Displayed: <script>alert(1)</script>

Code Explanation

- **Escapes HTML special characters**
- **Prevents script execution**
- **Protects against XSS attacks**

Task 5: SQL Injection Prevention

Question

Prevent SQL injection in database queries.

Prompt

"Generate Python SQLite code and secure it using parameterized queries."

Code

```
import sqlite3

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

cursor.execute("CREATE TABLE users (id INTEGER, name TEXT)")
cursor.execute("INSERT INTO users VALUES (1, 'Alice')")

user_input = input("Enter name: ")

query = "SELECT * FROM users WHERE name = ?"
cursor.execute(query, (user_input,))

print(cursor.fetchall())
```

Output

```
Enter name: Alice
[(1, 'Alice')]
```

Code Explanation

- Uses parameterized query (?)
- Prevents SQL injection
- Avoids string concatenation
- Ensures safe database access